

Program - 10

```
import numpy as np
import pandas as pd

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from imblearn.over_sampling import SMOTE

# Step 1: Create Imbalanced Dataset

X, y = make_classification(
    n_samples=1000,
    n_features=10,
    n_classes=2,
    weights=[0.95, 0.05],
    random_state=42
)

print("Original Class Distribution:")
print(pd.Series(y).value_counts())

# Step 2: Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# Step 3: Define Models

models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier()
}

# Function to evaluate models
```

```
def evaluate_model(model, X_train, X_test, y_train, y_test):
```

```
    model.fit(X_train, y_train)
```

```
    predictions = model.predict(X_test)
```

```
    acc = accuracy_score(y_test, predictions)
```

```
    print("\nAccuracy:", acc)
```

```
    print("\nConfusion Matrix:")
```

```
    print(confusion_matrix(y_test, predictions))
```

```
    print("\nClassification Report:")
```

```
    print(classification_report(y_test, predictions))
```

```
# Step 4: Evaluate on Imbalanced Data
```

```
print("\n===== IMBALANCED DATA =====")
```

```
for name, model in models.items():
```

```
    print("\nModel:", name)
```

```
    evaluate_model(
        model,
        X_train,
        X_test,
        y_train,
        y_test
    )
```

```
# Step 5: Apply SMOTE to Balance Data
```

```
smote = SMOTE()
```

```
X_train_balanced, y_train_balanced = smote.fit_resample(
    X_train,
    y_train
)
```

```
print("\nBalanced Class Distribution:")
```

```
print(pd.Series(y_train_balanced).value_counts())
```

```
# Step 6: Evaluate on Balanced Data
```

```
print("\n===== BALANCED DATA =====")
```

```
for name, model in models.items():
```

```
    print("\nModel:", name)
```

```
    evaluate_model(
        model,
        X_train_balanced,
        X_test,
        y_train_balanced,
        y_test
    )
```

output-

Original Class Distribution:

0 947

1 53

Name: count, dtype: int64

===== IMBALANCED DATA =====

Model: Logistic Regression

Accuracy: 0.9366666666666666

Confusion Matrix:

[[278 2]

[17 3]]

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.99	0.97	280
1	0.60	0.15	0.24	20
accuracy			0.94	300
macro avg	0.77	0.57	0.60	300
weighted avg	0.92	0.94	0.92	300

Model: Decision Tree

Accuracy: 0.95

Confusion Matrix:

[[273 7]

[8 12]]

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.97	0.97	280
1	0.63	0.60	0.62	20
accuracy			0.95	300
macro avg	0.80	0.79	0.79	300
weighted avg	0.95	0.95	0.95	300

Model: Random Forest

Accuracy: 0.9666666666666667

Confusion Matrix:

```
[[279  1]
 [ 9 11]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	1.00	0.98	280
1	0.92	0.55	0.69	20
accuracy			0.97	300
macro avg	0.94	0.77	0.83	300
weighted avg	0.97	0.97	0.96	300

Balanced Class Distribution:

0 667

1 667

Name: count, dtype: int64

===== BALANCED DATA =====

Model: Logistic Regression

Accuracy: 0.8866666666666667

Confusion Matrix:

```
[[251  29]
 [ 5 15]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.90	0.94	280
1	0.34	0.75	0.47	20

accuracy		0.89	300
macro avg	0.66	0.82	0.70 300
weighted avg	0.94	0.89	0.91 300

Model: Decision Tree

Accuracy: 0.93

Confusion Matrix:

```
[[265 15]
 [ 6 14]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.95	0.96	280
1	0.48	0.70	0.57	20

accuracy		0.93	300
macro avg	0.73	0.82	0.77 300
weighted avg	0.94	0.93	0.94 300

Model: Random Forest

Accuracy: 0.9333333333333333

Confusion Matrix:

```
[[267 13]
 [ 7 13]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.95	0.96	280
1	0.50	0.65	0.57	20

accuracy		0.93	300
macro avg	0.74	0.80	0.76 300
weighted avg	0.94	0.93	0.94 300